

A High-Quality Generation Approach for Educational Programming Projects Using LLM

Tian Song^{ID}, Member, IEEE, Hang Zhang^{ID}, and Yijia Xiao

Abstract—High-quality programming projects for education are critically required in teaching. However, it is hard to develop those projects efficiently and artificially constrained by the lecturers' experience and background. The recent popularity of large language models (LLMs) has led to a great number of applications in the field of education, but concerns persist that the output might be unreliable when dealing with intricate requirements. In this study, we design a customized role-based agent (CRBA), which can be configured for different roles specializing in specific areas of expertise, making the LLM yield content of higher specialization. An iterative architecture of multi-CRBAs is proposed to generate multistep projects, where CRBAs automatically criticize and optimize the LLM's intermediate outputs to enhance quality. We propose ten evaluation metrics across three aspects to assess project quality through expert grading. Further, we conduct an A/B test among 60 undergraduate students in a programming course and collect their feedback through a questionnaire. According to the students' rating results, the LLM-generated projects have comparable performance to man-made ones in terms of project description, learning step setting, assistance to students, and overall project quality. This study effectively integrates LLM into educational scenarios and enhances the efficiency of creating high-quality and practical programming exercises for lecturers.

Index Terms—Automatic generation, generative pretrained transformer (GPT), large language models (LLMs), programming education, programming projects.

I. INTRODUCTION

LEARNING material plays an important role in organizing a course [1]. Within the framework of programming education, assignments are regarded as pedagogical instruments that complement lectures by teaching the same material but in an exploratory fashion [2]. Specifically, programming projects require learners to address certain problems through coding, thereby developing programming skills that are goal-directed and well-organized behaviors cultivated through practice and effort [3]. This way, programming projects can scaffold the learning process, develop learners' mastery through regular practice, and provide immediate feedback.

Received 31 August 2023; revised 5 February 2024, 27 May 2024, 31 July 2024, and 5 November 2024; accepted 6 November 2024. Date of publication 15 November 2024; date of current version 3 December 2024. This work was supported by the National Natural Science Foundation of China under Grant 62077004. (Corresponding author: Tian Song.)

Tian Song and Hang Zhang are with the School of Education, Beijing Institute of Technology, Beijing 100081, China (e-mail: songtian@bit.edu.cn).

Yijia Xiao is with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China.

Digital Object Identifier 10.1109/TLT.2024.3499751

Learners vary in knowledge levels, but instructors cannot accommodate various learner levels due to the high cost of designing programming projects. Besides, learners demand novel and varied projects, which keep the learning process engaging and enhance their problem-solving skills. The quality of projects also greatly impacts the learning experience, for example, bug-ridden sample codes will confuse learners. The constraints on instructors' time and energy make it difficult for them to design such projects to fulfill student's needs. This makes the idea of automatically generating high-quality educational programming projects an exciting prospect.

Advances in generative AI and large language models (LLMs) have driven the development of many educational technology innovations that automate the time-consuming tasks of generating and analyzing text [4]. For example, code can be generated from natural language descriptions utilizing OpenAI Codex [5], enabling learners to concentrate more on theoretical and problem-solving aspects of computational thinking [6]. Various programming tasks can be solved by chat generative pretrained transformer (chatGPT) [7] powered by GPT-3.5 and GPT-4, such as conceptual understanding, essay evaluation, solution code generation, and error checking and debugging [8], [9].

There are studies focused on educational material generation utilizing LLMs. Sridhar et al. [10] generated learning objectives (LOs) based on a carefully crafted prompt involving detailed guidelines on high-quality LO authoring. Sarsa et al. [11] used an LLM to create programming exercises including sample solutions, test cases, and code explanations. Denny et al. [12] proposed a model called robosourcing where LLMs are in collaboration with humans to produce programming exercises. Leiker et al. [13] used LLMs for course creation without compromising accuracy or clarity. However, LOs in [10] are only simple text starting with an action verb, not covering complex logic like code. Sarsa et al. [11] only explored generating code snippets suitable for novice learners, such as an if-else function, and it requires detailed prompts including input-output parameters and specific function logic. Some methods require humans in the loop to assure high quality [12], [13].

Considering the above limitations, it is crucial to investigate how to utilize LLM to automatically produce high-quality educational programming projects. In order to facilitate lecturers' use, the approach should reduce the requirements for user input, and human intervention during the generation. Under such conditions, ensuring the quality of LLM-generated content is a major challenge. In this study, we first design a customized

role-based agent (CRBA), which can be configured with specific roles, tasks, and actions to produce content of high specialization. On this basis, an iterative architecture of multi-CRBAs (IAM-CRBA) is proposed, where criticism is introduced to ask for an LLM to take a hard look at its outputs. It can guide an LLM to automatically assess and enhance its outputs with the help of CRBAs, thus eliminating the noise of the LLM being used. Learner level and project requirements can be set by instructors to create their desired projects.

On the one hand, it is necessary to demonstrate the effectiveness of the proposed method; on the other hand, as the generated questions are expected to be employed in real teaching scenarios, an understanding of user experience is essential. Specifically, this study seeks to address the following research questions.

- 1) How can the proposed IAM-CRBA improve the quality of generated projects to a practical level?
- 2) What is the result of comparing LLM-generated projects using the proposed IAM-CRBA versus projects generated manually from the student's perspective?

To answer the first question, experiments are conducted with diverse parameter settings, and the utility of CRBA and the iterative architecture is indicated through comparative analyses. Besides, ten quality evaluation metrics are designed based on the characteristics of project components and educational needs and two projects are assessed through expert grading. As for the second question, an A/B test is conducted through questionnaires with 60 junior undergraduate students enrolled in the real course *Electrical and Computer Engineering (ECE) Internship* from the Beijing Institute of Technology.

Our main contributions are summarized as follows.

- 1) We design a CRBA, which enables an LLM to work independently as various agents and produce professional content.
- 2) We propose an IAM-CRBA, which enables CRBAs to iteratively optimize distinct parts of a programming project.
- 3) We conduct an A/B test with 60 junior undergraduate students to compare the man-made projects versus the LLM-generated ones, and their feedback is collected through a questionnaire for further analysis.

The rest of this article is organized as follows. Section II reviews the related literature. Section III introduces the details of the proposed method. Section IV demonstrates the proposed quality evaluation metrics and shows the rating results. Section V presents a user study and analyzes the student feedback. Finally, Section VI concludes this article and suggests directions for future research.

II. RELATED WORK

A. Educational Material Generation

There is some preliminary work using LLMs to automatically produce educational content, such as practice problem and quiz [14], [15], [16], [17], [18], [19]. Dijkstra et al. [15] proposed an end-to-end quiz generator EduQuiz using GPT-3 to generate multiple-choice questions and answers for a reading comprehension task. Lu et al. [16] proposed a system ReadingQuizMaker providing NLP-based process-oriented support to help educators

design high-quality questions. Blobstein et al. [17] introduced Angel to generate question-answer pairs of varied difficulty while targeting higher cognitive levels. Olney [18] compared LLM-generated questions to those from a college science textbook, finding no significant differences in performance on six out of seven measures tested. Since quiz generation is a relatively simple task, often requiring only input text and a specified template, it can produce high-quality output without the need for further tuning efforts.

LLMs are commonly used for code solution generation in the area of introductory programming. Piccolo et al. [20] found that ChatGPT can complete 75.5% of basic to moderate level programming exercises for an introductory bioinformatics course but require human feedback to solve more. Ouh et al. [21] confirmed that ChatGPT-generated solutions are effective when the instructions are straightforward, but the results may be incorrect when handling more complex instructions. Malinka et al. [22] evaluated ChatGPT across three types of programming assignments: small projects, term projects, and interactive projects, and revealed its good performance in the first two but unsatisfactory results in the last. This suggests that LLMs show poorer performance on more complicated code, which is inadequate for current research.

Learner resource is not confined to code in the context of programming education. Sarsa et al. [11] generated programming exercises, including problem statements, sample code solutions, and tests, using an LLM. However, only 30.9% of the generated solutions passed the tests. Denny et al. [12] introduced robosourcing for creating programming exercises using OpenAI Codex. Similarly, Leiker et al. [13] leveraged an LLM to generate courses, which accelerated the creation of educational content without compromising accuracy and clarity. However, the two methods include a human in the loop to provide adjustment for the model. To conclude, existing educational material generation strategies perform well on relatively simple tasks, such as quiz generation, but poorly on more complicated ones, with some requiring human intervention to ensure high quality. Our work will provide a method to automate the generation of high-quality programming projects using LLM without human intervention.

B. Prompt Engineering

Prompt engineering is an increasingly important skill set for its ability to program the LLM by customizing it and enhancing its capabilities [23]. Reeves et al. [24] found that small variations in prompts had a noticeable effect on the Codex's performance and the effect varied across different types of problems. Denny et al. [25] made prompts a strategy for students to resolve errors during programming, and showed that complex problems could be solved by engineering the prompts to contain explicit algorithmic hints. To encapsulate specific tasks and better leverage the LLM's capability, the idea of agent-oriented programming [26] can be introduced in prompt engineering. The agent, essentially comprising multiple organized prompts, defines an intelligent object, supports parameter configuration, and interacts with an LLM.

Since LLMs do not always generate optimal results on their first try, several studies consider multiturn prompt engineering to refine the output. Some studies [27], [28], [29] employed a supervised approach, but they may necessitate extensive training data or costly human annotations. Some research designed refinement approaches without extensive supervision. Shi et al. [30] proposed a three-turn iterative prompting, including task instruction, keyword-constrained prompt, and length-constrained prompt. Madaan et al. [31] introduced SELF-REFINE, in which the LLM provided feedback for its output and used it to refine itself iteratively. Gou et al. [32] implemented a framework called CRITIC allowing LLMs to validate and progressively amend their outputs with the help of external tools like code interpreters. Due to the fact that a comprehensive project involves multiple steps and facets, such as introduction, learning steps, sample code, and tests, this article adopts the SELF-REFINE approach and explores the crafting of effective prompts to guide an LLM in refining facets of a project.

III. METHODOLOGY

The section describes details of the proposed IAM-CRBA, whose goal is to automatically generate high-quality programming projects for learners for programming education.

A. Definition of Project

Regarding the difficulties found in the teaching–learning process of computer programming, one of the explanations is that learners lack the ability to solve problems despite being familiar with fundamental programming constructs [33]. Inspired by the cognitive apprenticeship model [34], which divides instruction into modeling, scaffolding, and fading stages, it is concluded that a good programming project for educational purposes should meet the following requirements.

- 1) The project gives learners a clear understanding of what knowledge they need to acquire.
- 2) The project helps learners develop mastery of the knowledge through worked examples.
- 3) The project offers guidance to learners in accomplishing a task or even delving deeper into the task.
- 4) The project provides immediate feedback to learners for building learners' comfort levels [35].

Sarsa et al. [11] defined that programming exercises involve creating problem statements, sample code solutions, and automated tests. The programming project mentioned in this article is essentially a form of programming exercise, and its composition is extended to better facilitate student learning in an educational context. Combined with the above needs, it is defined that a comprehensive project involves three parts: introduction, implementation, and expansion. The expansion is optional and facilitates customization based on the instructor's preferences. The composition of the project is shown in Fig. 1, in which the specific parts of expansion are just examples.

The introduction involves an overview of the project. For example, there is a project on operating the Redis database using Python, and the introduction will briefly explain what Redis is and which Redis commands can be learned in the

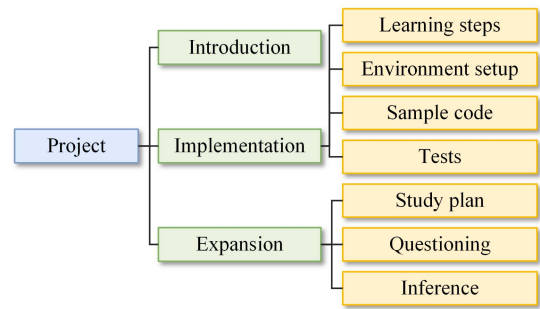


Fig. 1. Composition of a complete project.

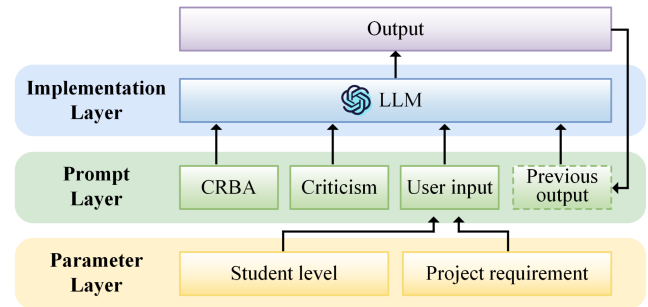


Fig. 2. Proposed three-layer prompt architecture.

project. This allows learners to have a general view of a project. Implementation is the main body of the project, consisting of learning steps, environment setup, sample code, and tests. The project is divided into distinct steps, each corresponding to a specific task. Compared to no steps, setting steps helps learners establish a clear learning path and makes it easier to achieve LOs. In this context, the learning steps include first connecting to the local Redis service, then using the SET command, and finally using the GET command to check if the SET operation is successful. Corresponding sample codes and tests are provided under each task. Environment setup offers key commands for configuring the environment. Expansion is used to further enhance the learning experience. For instance, a study plan based on the Pomodoro technique can be added to facilitate learners in effectively managing their time.

B. Prompt Architecture

Prompts are instructions given to an LLM to enforce rules, automate processes, customize the output, and ensure specific qualities of the output [23]. An LLM generates content based on prompts, so it is crucial to formulate effective prompts that drive informative conversations. To automatically generate a high-quality programming project, a three-layer prompt architecture is proposed, as shown in Fig. 2.

The first layer is the parameter layer, which supports user inputs, including student level and project requirements. Instructors can select a suitable level based on the student's class performance, while evaluating students' project performance reflects their levels, giving feedback to the model input. The project requirement can be concise, such as "Design an image processing project using Python that can achieve image color

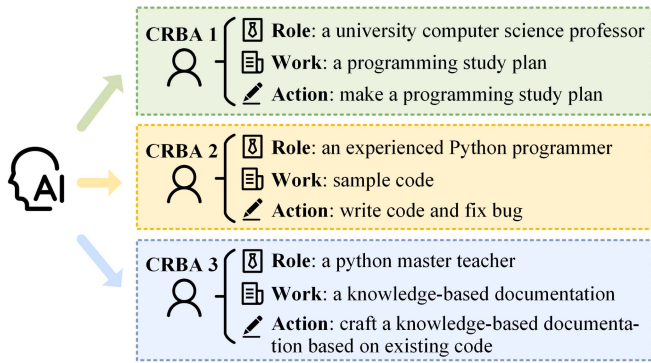


Fig. 3. Example of CRBAs designed for generating a Python programming project targeted at college students.

transformation.” Since an LLM is trained on a massive corpus of text and code, it can generate a comprehensive project based on such a brief description.

The second layer is the prompt layer, including four types of prompts: user input, agents, criticism, and previous output. Some research showed that one of the most effective techniques for crafting effective ChatGPT prompts is the *act as* hack [36]. Inspired by this, we propose CRBA, as detailed in Section III-C. Given that outputs in a single pass may be unreliable, criticism is introduced. For example, we ask the LLM: “Does the code fulfill the requirements of learning steps? Analyze bugs in the code.” Then, its response and the prior output are used as new input, with instructions to “Use the criticism to rewrite the code and ensure the code is executable.” Criticism enables the LLM to take a hard look at its outputs and further improve them without human intervention. The third layer is the implementation layer, an LLM, which generates content based on prompts.

C. Customized Role-Based Agent

CRBA is designed and implemented to be flexibly configured for different roles specializing in specific areas and used to execute the tasks aligned with their strengths. Each CRBA comprises three components: role, work, and action. Role corresponds to a profession characterized by specific levels of expertise. Work contains tasks to be accomplished, while action denotes the specific execution of the tasks.

As shown in Fig. 3, there are three CRBAs designed for creating a Python programming project targeted at college students. Take CRBA 1 for example, it is a university computer science professor, whose work is to write a programming study plan based on the known student level and project requirements. The reason for this assignment is that, acting as someone with extensive teaching experience at university, a professor has excellent skills in devising study plans for students. These skills are particularly specialized compared to those of other professionals, such as programmers, who focus on different fields.

Fig. 4 shows a code sample of the CRBA implementation, where *CRBA* class serves as an abstract template, setting rules for its subclasses to follow. The *ProjectPlanner* class is a specific subclass that inherits the rules and provides the actual implementation. The *_hasWorkToDo* method checks if there is

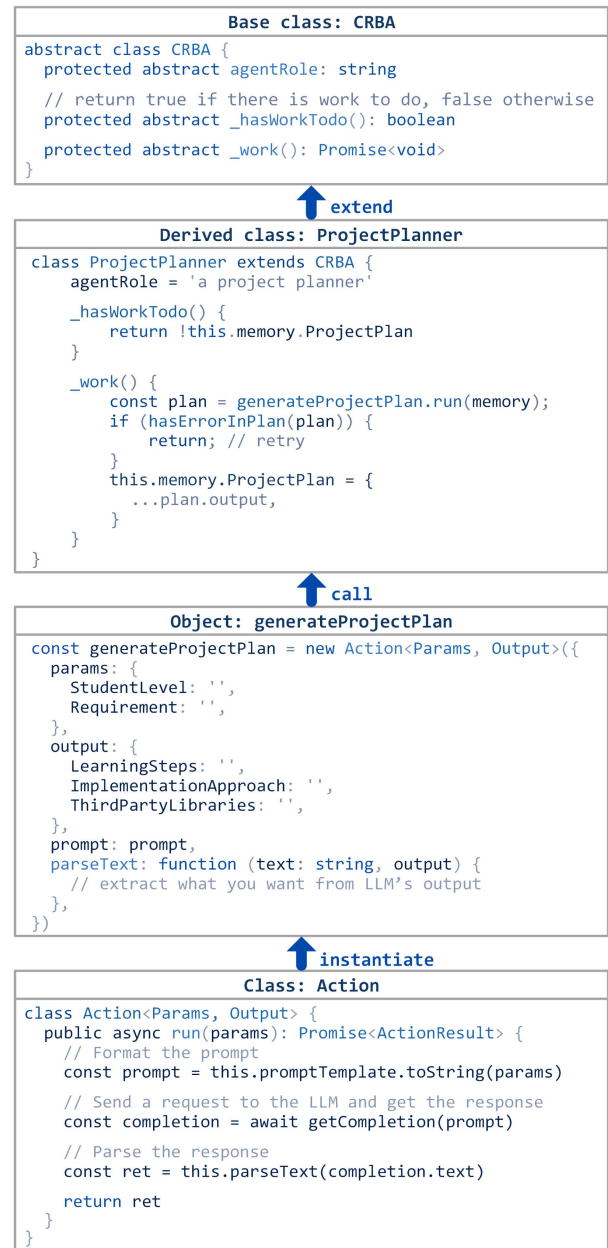


Fig. 4. Critical code of implementing CRBA.

any work to do. The *_work* method calls the *run* method of *generateProjectPlan* object to create a study plan. Fig. 5 shows the prompts used for initializing *generateProjectPlan*. Guided by such prompts, an LLM simulates itself as an experienced teacher, configures the knowledge and experience for this role, and generates content.

The concept of modular thinking inspires us to propose CRBA. Since a complete project encompasses diverse types of content, it is natural for us to consider dividing it into distinct works and utilizing distinct roles for execution. The efficacy of using CRBA is threefold. First, guiding the LLM to act as specific roles drives greater engaging and informative interactions, making it yield content of higher specialization. Second, customizing CRBAs is convenient to tailor to the individual

```

Role: You are Sophia, a university computer science professor. You are going to
design a programming study plan for {StudentLevel} students based on
{Requirement}.
Format: According to the context, fill in the following missing information. If the
requirements are unclear, ensure minimum viability and avoid excessive design.
Attention: Use '##' to split sections, not '#'. And '## <SECTION_NAME>' should
write before the code and triple quote.
## Learning Steps
Provide as a single-level list. Up to THREE clear and actionable steps, each
focusing on the realization of a small function. Ensure that {StudentLevel}
students are able to complete them.
## Implementation Approach
Provide as plain text. Describe the overall implementation approach, including
the design pattern, the third-party libraries used and the reasons for using them.
## Third-Party Libraries
Provide as a list. Third-party libraries that are helpful to implement this project.

```

Fig. 5. Example of prompt template.

interests of the user and the specific requirements of the project. Third, the design of task division and arrangement helps to enhance the content quality, manageability, and scalability of the generation process.

D. Iterative Architecture of Multi-CRBAs

LLMs do not always generate the desired results on their first try. Studies show that LLMs tend to fall short in addressing intricate requirements, for instance, tasks with multifaceted objectives, such as dialogue response generation [31]. To improve output quality, refinement is required. Leveraging their working memory, LLMs can revisit prior outputs, guided by prompts that encourage self-critique and feedback. IAM-CRBA is proposed to automatically assess and refine the intermediate outputs of the generative AI model.

Different from previous work, the assessment is based on CRBAs, making it more specialized. For example, a university teacher considers optimizing a project from a teaching perspective to better serve educational purposes. Meanwhile, a course sales perspective aims to refine the project to make it more appealing to potential purchasers. Compared to the work in [31] that directly provides feedback on the final results, our refinement occurs at various stages of the entire generation process. Solving a complex problem involves multiple interdependent steps. Thus, the LLM is guided step by step, ensuring each step's result is optimal before moving on.

An example of IAM-CRBA is shown in Fig. 6, with only two examples of the iterative process depicted due to space constraints. Our goal is to generate an image preprocessing project using Python aimed at novice learners. CRBA 1 is an experienced computer teacher, CRBA 2 is a Python programmer, and CRBA 3 is a course sales representative. First, CRBA 1 crafts several learning steps suitable for novices based on the requirements. Next, CRBA 2 writes code following the learning steps, assesses the code from multiple perspectives, such as integrity and correctness, and determines whether to refactor it. The process iterates until a specific termination condition is met. Finally, CRBA 3 refines the project to make it more appealing to students. After N rounds of refinement, the ultimate project is obtained.

The details of one iteration on a test file are illustrated in Fig. 7. The CRBA reflects on the first version according to the reflection

TABLE I
USER INPUT OF THE REDIS PROJECT

Parameters	Description
Student level	Intermediate
Project requirement	Using the Go language to build a toy Redis clone capable of handling basic commands.

perspectives for CRBA 2, as shown in Fig. 6, and finds out that this file only checks if the main file can successfully compile without testing for the Redis data structure. Since the prompt includes the question “*Should you rewrite the code according to the criticism?*,” the CRBA determines to refactor the code based on its criticism. Next, the CRBA makes code changes resulting in the second version of the test file and that is one round of iteration. Since the second version achieves the requirements, the iteration ends.

To explore the impact of iteration on the output quality, a project is generated with the user input outlined in Table I and we set the number of iterations from zero to four. Fig. 8 shows the results regarding the introduction and learning steps where text highlighting in green indicates differences between the versions. The results are referred to as version-0 to version-4 for clarity. Version-3 and version-4 are not shown because, compared with version-2, they do not offer effective information but only include minor changes such as format. As for the project introduction, version-1 has three differences compared to version-0. First, it specifies that the programming language is Go. Second, it adds specific operations to be mastered in this project. Third, there are some supplements. For example, as Redis supports multiple data types, here the focus is on string data type. In version-2, the input and output forms of the project are further detailed.

In terms of the learning steps, in version-1, the *store and retrieve* step is split into two distinct steps because they essentially serve different purposes. Besides, explicit needs are incorporated like *string data type* and *in seconds*. In version-2, detailed requirements for the input and output parameters of all functions are established. In version-3 and version-4, the project introduction remains almost unchanged. As for the learning steps, in version-3, there are supplements regarding when to return an error message, such as specifying in the third step that *the function should return an error message if the key does not exist in the database*. In version-4, there are examples of the input and output of functions, such as *db.Delete(“name”) returns “key deleted successfully.”*

IAM-CRBA represents a process of supplementation and refinement in essence. Iterations generally follow the initial version without making significant changes. In this example, the learning steps in the three versions are almost the same. The iterative process provides a supplement to previous rounds, such as data types, data units, and error handling. However, as the number of iterations increases, the outputs tend to already meet the user requirements. Although each iteration only takes a few seconds, having little impact on user experience, it requires more tokens for application programming interface (API) calls, leading to increased costs. Therefore, if additional iterations

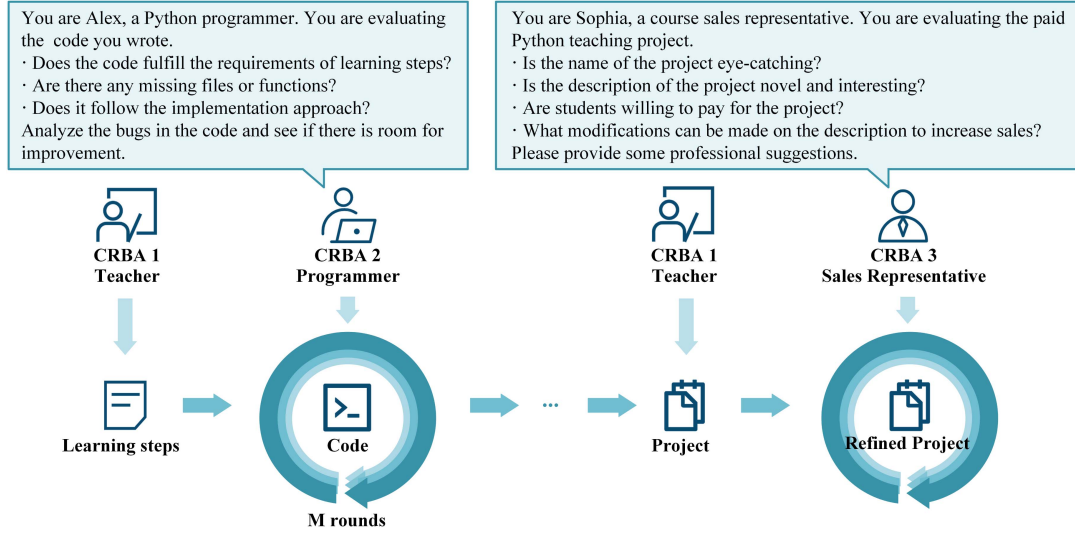


Fig. 6. Iterative architecture of multi-CRBAs.

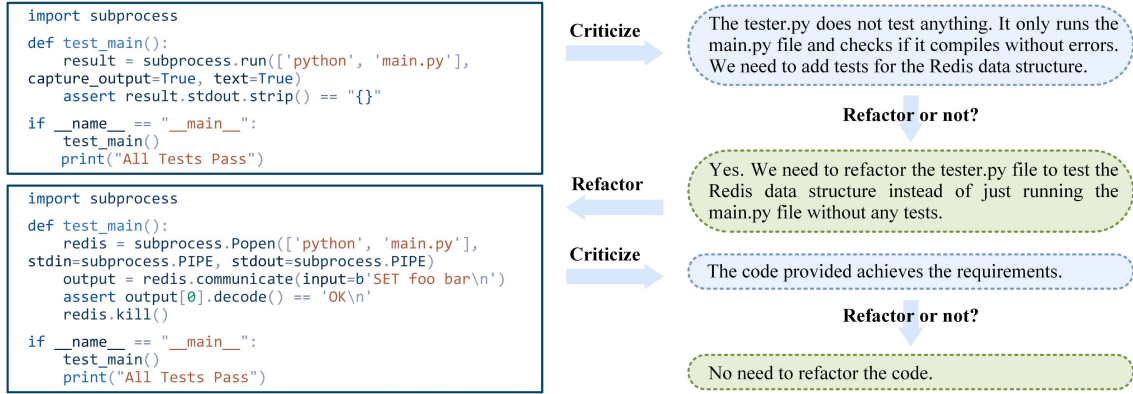


Fig. 7. Example of one round of an iterative process.

merely refine less important details or deviate from the initial requirements, it is advisable to terminate the iterative process.

The use of IAM-CRBA is flexible and customizable, and based on our experiments, three key points have been identified to maximize the proposed architecture's potential. First, the need for iteration depends on task complexity and the desired quality. If there is limited potential for improvement, it is not necessary to establish an iterative process. Second, while iteration can enhance output quality, excessive iteration incurs extra costs without leading to better outcomes. Third, different CRBAs have distinct optimization goals, so the choice of CRBA influences results. For the same project, having a course sales representative optimize from a selling perspective will yield different results than having a computer teacher optimize from an instructional standpoint.

IV. EXPERIMENTAL EVALUATION

This section addresses the first research question, *How can the proposed IAM-CRBA improve the quality of generated projects?*, by detailing experiments that utilize GPT-3.5. First, it is shown that different learner levels have distinct effects on the generated projects through a typical example. Second, the utility of CRBA

is verified, and a deeper exploration into configuring effective descriptions for CRBA is conducted. Third, the project quality is assessed with the proposed metrics.

A. Impact of the Learner Level

1) *Experimental Setup*: In order to demonstrate how different learner level parameters lead to projects of varying difficulty, comparative experiments are conducted, and learners are categorized into three levels: novice, intermediate, and advanced learners. Since our focus is on facilitating instructors in generating exercises, students are roughly categorized into groups for different exercise levels. The choice of the three levels is for convenience in presenting and contrasting experimental results, but can be determined by instructors in practice. Apart from this parameter, all other configurations are the same. The basic requirement for generating this project is *design an image processing project using Python to familiarize learners with the usage of the OpenCV library*.

2) *Results*: As the difficulty of learning steps can reflect a project's overall difficulty, only the aspects are compared here to facilitate the writing. Fig. 9 illustrates the learning steps generated for three different learner levels. For novices, except

Project Introduction

Welcome to the programming teaching program! In this project, we will be building a toy Redis clone capable of handling basic commands. Redis is an open source, in-memory data structure store, used as a database, cache, and message broker. Our clone will not have all the features of Redis but will be a great starting point to learn about databases and how they work.

Learning Steps

1. Create a struct to represent the database, with a map to store the key-value pairs.
2. Implement a function to store and retrieve key-value pairs in the database.
3. Implement a function to delete a key-value pair from the database.
4. Implement a function to check if a key exists in the database.
5. Implement a function to list all the keys in the database.
6. Add an expiration time field to the struct, and modify the functions to handle expiration times.

(a)

Project Introduction

Welcome to this exciting programming teaching program! In this project, we will be building a simplified version of Redis, a popular in-memory data structure store, using Golang. Our version will be able to handle basic commands such as storing and retrieving key-value pairs, deleting key-value pairs, checking if a key exists, listing all keys in the database, and adding an expiration time to a key-value pair. This project will provide a great opportunity to learn about databases, data structures, and time management in programming. The key-value pairs that can be stored in the database will be of string data type, and the expiration time will be represented in seconds.

Learning Steps

1. Create a struct to represent the database, with a map to store the key-value pairs of string data type.
2. Implement a function to store and retrieve key-value pairs in the database.
3. Implement a function to delete a key-value pair from the database.
4. Implement a function to check if a key exists in the database.
5. Implement a function to list all the keys in the database.
6. Add an expiration time field in seconds to the struct, and modify the functions to handle expiration times.

(b)

Project Introduction

Welcome to this engaging programming project! We will be creating a simplified version of Redis, a widely used in-memory data structure store, using Golang. Our version will be capable of handling basic commands such as storing and retrieving key-value pairs, deleting key-value pairs, checking if a key exists, listing all keys in the database, and adding an expiration time to a key-value pair. This project will offer a fantastic opportunity to learn about databases, data structures, and time management in programming. The key-value pairs that can be stored in the database will be of string data type, and the expiration time will be represented in seconds. We will also provide detailed information on the input and output forms of the project, especially for the functions that store, retrieve, delete, and list key-value pairs.

Learning Steps

1. Create a struct to represent the database, with a map to store the key-value pairs of string data type.
2. Implement a function to store key-value pairs in the database. The function should take a key and a value as input and store them in the database.
3. Implement a function to retrieve a key-value pair from the database. The function should take a key as input and return the corresponding value.
4. Implement a function to delete a key-value pair from the database. The function should take a key as input and remove the corresponding key-value pair from the database.
5. Implement a function to check if a key exists in the database. The function should take a key as input and return a boolean indicating whether the key exists.
6. Implement a function to list all the keys in the database. The function should return a list of all keys in the database.
7. Add an expiration time field in seconds to the struct, and modify the functions to handle expiration times. The function to add an expiration time should take a key and a time in seconds as input.

(c)

Fig. 8. Comparison of introduction and learning steps generated with different iteration numbers. (a) Results without iteration. (b) Results after one iteration. (c) Results after two iterations.

for the third step, the rest are the most basic image processing operations. For intermediate learners, tasks, such as facial recognition, are included, requiring a higher level of programming skills. For advanced learners, it enriches some functionalities of the other two versions. For example, while the novice one only requires saving images in PNG format, different formats are required here, which encourages learners to delve deeper into the distinctions between different image formats. It also introduces complex tasks, such as object tracking, requiring a deeper understanding of relevant techniques.

It can be concluded that setting different learner levels directly impacts the difficulty of the generated projects. Higher levels lead to more challenging projects. Research indicates that it is important for teachers to design self-directed tasks, such as homework in ways that allow students to experience success, as this supports and enhances student engagement. [37]. In a real education scene, a classroom often consists of students with

varying levels of expertise. Instructors can group students by their skill level and assign programming projects of appropriate difficulty. Since the exercises match students' proficiency, students are more likely to feel a sense of accomplishment as they successfully complete them, fostering enthusiasm and engagement in programming learning.

B. Impact of CRBA

1) *Experimental Setup*: The first task is to verify that different CRBA configurations yield different results. First, two CRBAs, assuming the roles of *experienced JavaScript teacher in a university* and *senior JavaScript programmer in an IT firm*, respectively, are employed to write code based on given learning steps. They are chosen for high representativeness. A study [38] found that high failure rates and difficulties in comprehending programming concepts and structures make learning to program

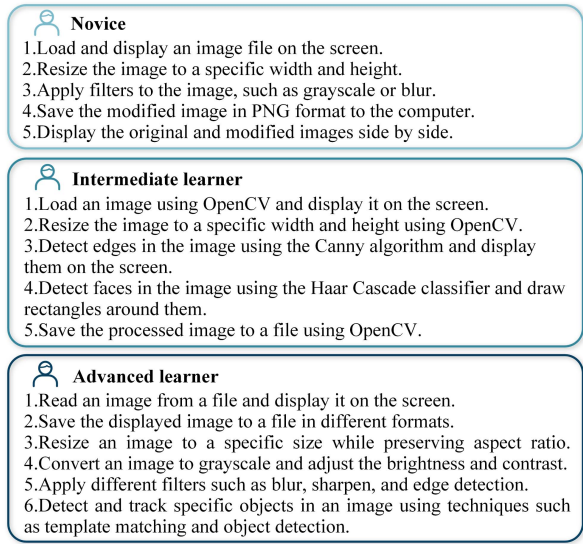


Fig. 9. Learning steps are generated according to different learner levels.

TABLE II
SETTINGS OF ROLES IN THE THIRD TASK

Id	Description
1	A computer science teacher.
2	An experienced computer science teacher.
3	A computer science teacher with 20 years of teaching experience.
4	A computer science master teacher and head of a teaching team.

challenging for novices. In educational contexts, clarity and readability are emphasized to facilitate learning, especially for novices. Conversely, professional programmers focus more on performance and maintainability due to practical constraints and long-term project goals. Next, two CRBAs, assuming the roles of *computer science professor in a university* and *online course sales representative*, respectively, are used to provide another project design referring to the existing one for testing students’ learning outcomes.

It is well known that the *acts as* command for AI prompts is helpful in generating high-quality results. What we wonder is whether the results differ when the descriptions of the same role are different. Our second task investigates effective descriptions for CRBA prompts. Specifically, learning steps generated by four CRBAs—all designated as computer science teachers but with varying descriptions of experience as given in Table II, referred to as role-1 to role-4, are compared. Experiments are carried out by generating a project named Match-3-Game with fixed parameters outlined in Table III. To ensure the validity of the experimental results, all settings remain identical except for the role of CRBAs, and the number of iterations is set to one here.

2) *Results*: To keep the writing concise, only the code for the first learning step, as shown in Fig. 10, is analyzed. It can be seen that the most significant difference between the two segments is that the programmer applies the principle of object-oriented programming and model-view-controller (MVC) architecture.

TABLE III
USER INPUT OF THE MATCH-3-GAME PROJECT

Parameters	Description
Student level	Intermediate
Project requirements	Build a Match 3 game using PIXI.js. There will be various gemstones of different colors on the game screen. The rule of the game is to connect three gemstones of the same color by exchanging two adjacent gemstones to eliminate them.

While the teacher uses procedural programming where all functionalities are implemented within the same function. In addition, when creating gemstone sprites with different colors and positions, the teacher imports background images and specifies their positions manually one by one. The programmer uses random numbers to achieve both random colors and positions without manual specification.

It cannot be judged which code instance is good and which is bad without assessment based on subjective criteria. What is clear is that, in this example, the code generated by the teacher is fairly straightforward to grasp for learners but lacks scalability. The code generated by the programmer has great maintainability and scalability, yet requires a higher level of foundational knowledge and is harder to understand. The choice of which code style to adopt depends on multiple facets, such as the learner’s ability, teaching design, and course objectives. For instance, if students already have certain JavaScript programming experience and the course is related to web development, it is a good choice to introduce structures, such as MVC teaching students, to build a scalable application.

Next, the project names and descriptions generated by another two CRBAs, as shown in Fig. 11, are compared. Both project names are eye-catching and bring forth the theme. The sales representative’s introduction highlights the skills that learners can acquire through the project, such as problem-solving abilities and critical thinking. From a sales perspective, stating the benefits can stimulate learners’ desire to purchase the project. The teacher’s version focuses on specific tasks and key points to be addressed, starting with basic requirements and encouraging deep thinking about some additional features. Essentially, it reflects a typical learning process, progressing from basic to advanced knowledge through increasingly challenging learning paths. Each version has its advantages and the selection depends on actual needs.

In the second task, each CRBA generates five learning steps. Given that role-2 and role-3 produce almost identical outputs, Fig. 12 compares the results written by roles-1, role-2, and role-4, where text highlighting in green is to indicate differences. The steps generated by role-1 are the most concise, and role-2 adds the condition of eliminating gemstones. The steps generated by role-3 are the most extensive and detailed. First, it shows thoughtful consideration regarding functionality, taking into account the game’s auto-ending and the highest score features that the other roles overlook. Next, it delves deeper into the triggering conditions of specific functions from an implementation perspective. Third, it explicitly

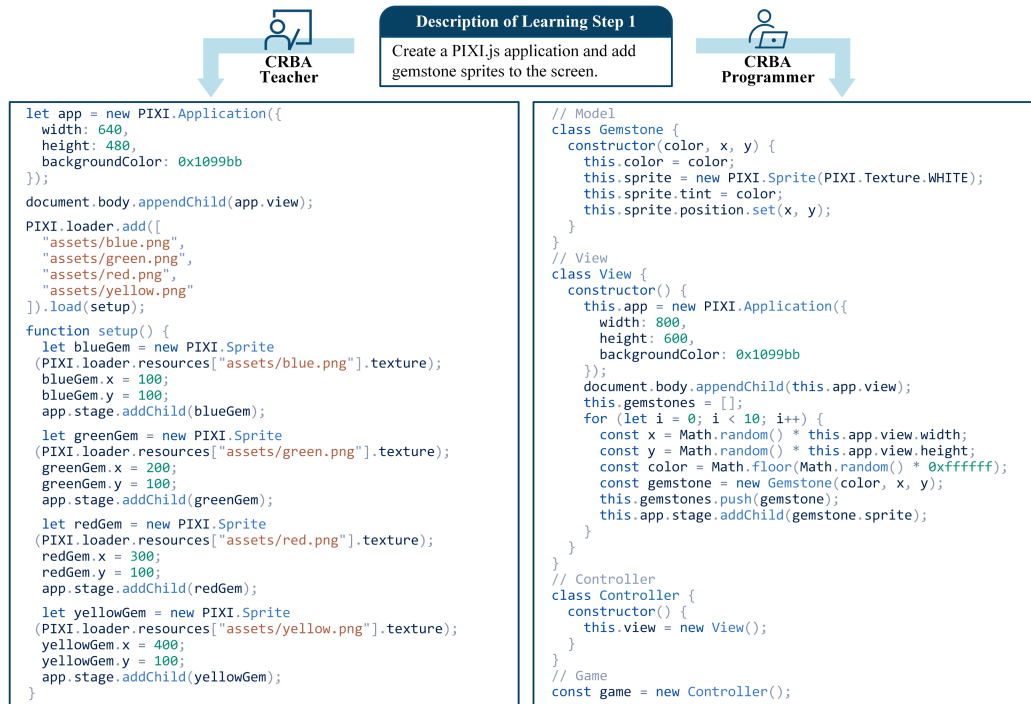


Fig. 10. Code generated by two CRBAs with different roles based on the description of the first learning step.

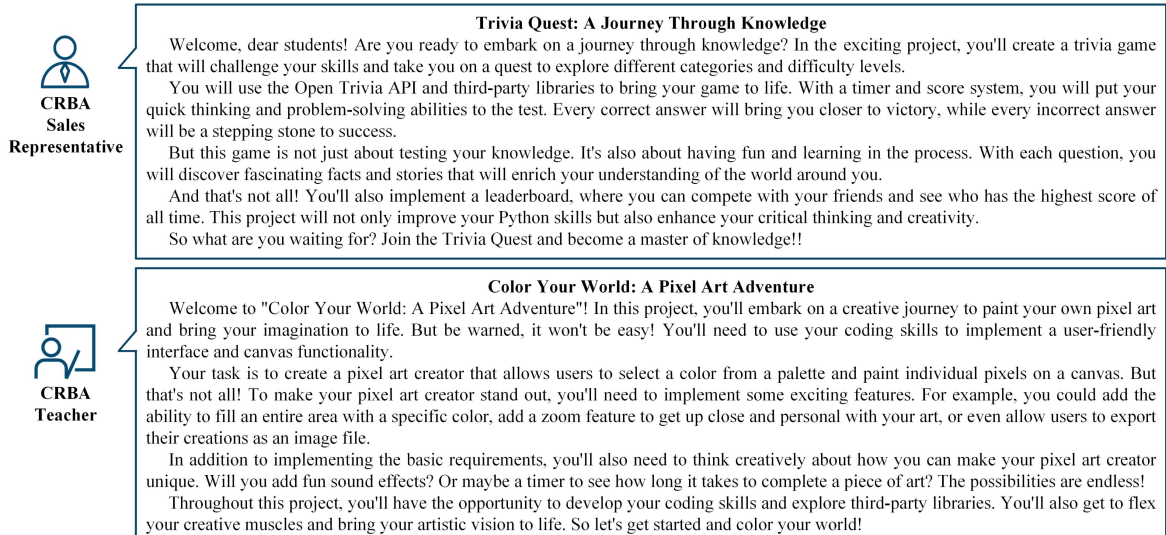


Fig. 11. Project names and descriptions generated by two CRBAs with different roles.

outlines the method or structure used to implement a certain function.

According to the experimental results, it is concluded that having an LLM act as various CRBAs yields different outcomes. Furthermore, the results generated by a specific CRBA tend to reflect the occupational characteristics of the given role. In a real education scene, instructors can customize CRBAs based on their needs along with the characteristics of different roles. A detailed description of the CRBA is recommended, such as describing the role as someone with "rich experience working in server-side Java development." However, whether there exist

universally effective descriptions for most roles requires further exploration and is left for future work.

C. Quality Evaluation

1) *Evaluation Metrics*: Unlike most LLM-generated content in existing studies, which consists of a single type of content, a project contains multiple factors, such as introduction, learning steps, sample code, and tests, each with unique features. Thus, we assess projects from three aspects and propose specific metrics for each, as given in Table IV. The first aspect is an overall

	 computer science teacher	 experienced computer science teacher	 computer science master teacher and head of a teaching team
Step 1	Create a game screen with PIXI.js and display different colored gemstones on it.	Create the game screen with PIXI.js and display different colored gemstones randomly on it.	Setting up the game screen with PIXI.js and displaying gemstones of different colors using a data structure.
Step 2	Implement the ability to swap two adjacent gemstones by clicking on them.	Implement the ability to select two adjacent gemstones on the screen and swap their positions.	Implementing the ability to swap two adjacent gemstones with mouse click or touch events and processing user input.
Step 3	Create a scoring system and update the score when gemstones are eliminated.	Check if three or more same colored gemstones are connected and eliminate them from the screen. Update the score accordingly.	Writing game logic to eliminate matched gemstones and calculate scores using a data structure.
Step 4	Create a countdown timer and decrement the remaining time as the game is played.	Implement the timer and display the remaining time on the screen.	Implementing a game over condition when there are no more possible moves.
Step 5	Implement the ability to restart the game when the player loses.	Add a restart button to reset the game.	Displaying the player's score and highest score achieved, and implementing a restart button using HTML and CSS.

Fig. 12. Learning steps generated by CRBAs with different descriptions of roles.

TABLE IV
QUALITY EVALUATION METRICS

Metrics for the overall	Definition
Complete	The project contains all the required components.
Instructive	The project can offer a lot of useful information to learners.
Metrics for the text	Definition
Clear	The texts are easy to understand and are not misleading.
Coherent	The texts exhibit excellent contextual coherence.
Relevant	The texts are closely correlated with the project input.
Metrics for the code	Definition
Correct	There are no bugs in the code.
Concise	There are no redundancies in the codes.
Appropriate name	The variables, functions, or classes have clear or conventional names.
Necessary comments	The codes provide necessary comments.
Relevant	The codes match the project input and the text in the project.

TABLE V
DIVISION OF PARTICIPANTS AND THE ASSIGNED PROJECT

Project type	Redis	Match 3 game
Man-made	30	30
LLM-generated	30	30

evaluation. Regardless of a project's quality, its fundamental requirement is to be *complete*. Since the generated content serves the field of education and should be appropriate for educational purposes, the *instructive* metric is introduced. The second aspect is to evaluate the text part, including introductions and learning steps, and the *clear*, *coherent*, and *relevant* metrics are proposed, referring to studies [11] and [39]. The third aspect is code evaluation. We reference clean code proposed by Martin [40] and propose the *correct*, *concise*, *appropriate name*, *necessary comments*, and *relevant* metrics. As expansion is an optional part of a project, it is not assessed here. All metrics are measured on a scale of 0–10 where zero represents completely dissatisfying the definition, and ten means completely satisfying the definition.

2) *Experimental Setup*: First, it is demonstrated that the proposed approach and generated projects are applicable to users. In our experiments, it takes 2 min on average for GPT-3.5 to generate a project containing three to four steps using our

method. A system has been implemented for instructors to publish projects and for students to complete them. The user interface is shown in Appendix A. Two projects created with the IAM-CRBA were evaluated by experts using the proposed metrics. The user input for the two projects is outlined in Tables I and III, respectively. In the following text, we refer to the first project as the Match 3 game project and the second one as the Redis project. The two projects are chosen for the following reasons. The Redis project is to complete basic commands for operating Redis, which are definite operations, while the Match 3 game project requires implementing a game with complex logic and interaction. These projects allow us to assess LLM performance on tasks of varying types and difficulties.

In both projects, a CRBA with the role description *a university computer science teacher with 20 years of teaching experience* is utilized to generate project introduction and learning steps, with one iteration. For the Redis project, a CRBA with the role description *an experienced Go programmer* is used to write, criticize, and refactor its code, with two iterations. For the Match 3 game project, the same coding work is carried out by a CRBA with the role description *an experienced JavaScript programmer*, with one iteration. Ten experts are invited for grading, including two professors, three engineers, and five instructors, all of whom are in the field of computer science and from the Beijing Institute of Technology.

These expert panel members are selected to ensure a well-rounded assessment. First, university professors have a deep understanding of theoretical concepts and pedagogy, ensuring that exercises contribute to learners' comprehension of programming principles. Second, the involved engineers are proficient in coding and can bring a practical and industry-oriented perspective to the evaluation. Since the programming project is for

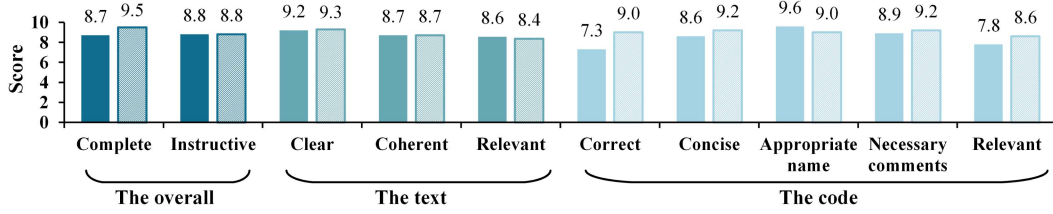


Fig. 13. Average quality scores. The left bar corresponds to the Match 3 game project and the right bar corresponds to the Redis project.

TABLE VI
STATISTICAL DATA OF THE QUESTIONNAIRE RESPONSES

Question	Project	Group	Mean	Std	Shapiro–Wilk			Mann–Whitney U	Wilcoxon W	Asymp. Sig. (2-sided)
					Stat	dif	Sig.			
1	Redis	Man-made	3.200	1.095	0.916	30	0.021	329.500	794.500	0.072
		LLM	3.933	0.907	0.839	30	<0.001			
	Match 3 game	Man-made	3.767	0.817	0.861	30	0.001	442.500	907.500	0.911
		LLM	4.000	0.947	0.846	30	<0.001			
2	Redis	Man-made	3.367	0.964	0.916	30	0.002	342.000	807.000	0.082
		LLM	3.733	0.907	0.904	30	0.010			
	Match 3 game	Man-made	3.200	0.925	0.863	30	0.001	468.500	933.500	0.770
		LLM	3.533	0.819	0.830	30	<0.001			
3	Redis	Man-made	2.933	0.980	0.905	30	0.011	364.000	829.000	0.181
		LLM	3.267	0.828	0.772	30	<0.001			
	Match 3 game	Man-made	3.067	0.868	0.894	30	0.006	359.000	824.000	0.148
		LLM	2.900	1.029	0.801	30	<0.001			
4	Redis	Man-made	5.065	2.407	0.958	30	0.283	283.000	748.000	0.010
		LLM	6.448	2.131	0.935	30	0.066			
	Match 3 game	Man-made	5.484	2.234	0.954	30	0.022	378.500	843.500	0.263
		LLM	5.600	2.762	0.917	30	0.222			

lecturers to assign to students, their evaluation is crucial to us. The experts were provided with the name and specific definition of each metric, as given in Table IV. Experts were gathered in a classroom and given 30 min to independently assign scores based on a ten-point Likert scale, where each point reflects a specific degree of compliance. This allows experts to assess the extent to which the generated content meets the provided metric definitions. During this period, experts were instructed not to discuss their assessments with each other, ensuring that each score was an independent judgment. Besides, tools were introduced for assessing code correctness. The code of the Match 3 game project written in Pixi.JS is tested in Visual Studio Code, and the code of the Redis project written in Go is tested in the Go Playground. Finally, the data are analyzed to demonstrate the project quality.

3) *Results*: Statistical analysis was conducted on the scoring results, as depicted in Fig. 13, where the means are calculated for each metric. The left one corresponds to the Match 3 game project while the right one corresponds to the Redis project. First, the scores of all metrics are above eight points except for the code correctness and relevance of the Match 3 game, indicating that the projects generated using the IAM-CRBA largely meet the metrics. For the Match 3 game project, the *correct* metric only scored 7.3, and the *relevant* metric scored 7.8. The experts were interviewed to gather their detailed feedback and to explore potential improvements for the projects. The open question was: “In your opinion, what are the deficiencies or areas for improvement in this project?” As for the Match 3 game project, the first expert pointed out that there are logical issues in the code

and the implementation of code in two out of the five learning steps deviates from the project requirements. The second expert stated that there is limited information on the third-party libraries and functions to be used. The third expert considered that some reusable code is not written in the form of functions, leading to a certain degree of redundancy. In summary, the Match 3 game has some logical issues and running errors and can be improved in terms of information adequacy and code simplicity.

Since an ideal project fully meets all metrics, a benchmark of ten points is used here. Comparing the two projects, the average score of the first one is 8.6, slightly lower than that of the second one. Concerning code generation, it adheres well to basic coding requirements, including naming conventions and necessary comments. According to our test results, the Redis project’s code runs successfully in the Go Playground. However, the Match 3 game project’s code can display and swap gemstones but fails to eliminate them, consistent with the expert’s explanation of his rating. That is because the former project is simpler and its logic is straightforward, while the latter one involves complex tasks, such as considering the position and color relationships of gemstones, and adding and eliminating gemstones, posing challenges for LLM. As for the code conciseness, the LLM occasionally complicates the matters. For instance, in the Redis project, unnecessary structures are used to store key–value pairs.

Based on expert scoring, the generated projects perform well on most metrics. However, in the fifth step of the Redis project, GPT-3.5 deviated from the original step of *implement a function to flush the data store to implement increment functionality*.

Besides, it does not always understand our intentions if details are insufficient. For example, in the Match 3 game project, the requirement to swap two adjacent blocks was interpreted differently under two calls: once as swapping two gemstones when selected successively by a user, and once as automatically swapping a selected gemstone with an adjacent one. Since the internal mechanisms of an LLM are unknown, the reasons behind it cannot be further analyzed. But what is certain is that clear and explicit functional descriptions help in generating the intended functionalities.

V. USER STUDY

A. Experimental Setup

In total, 60 junior undergraduate students from the Beijing Institute of Technology enrolled in the course ECE Internship participated in our experiment. They are randomly divided into two groups, each comprising 30 individuals, and assigned two projects, as given in Table V. Both man-made projects and LLM-generated ones are provided for comparison. Details of the man-made projects are given in Appendix B. Since our aim is to strengthen students' programming skills, sample code was not provided with the assignments and the focus here is on evaluating the text part of the generated projects. Students were asked to complete the two projects independently within 10 days. To ensure the validity of the experimental results, students were not informed about the involvement of AI. Students were treated equally in the experiments, with only variations in the assigned projects, and no personal characteristics, such as gender or grade, were involved.

The questionnaire consists of five questions. The first four questions use a Likert scale, where each presents a statement with five options: *strongly agree*, *agree*, *neutral*, *disagree*, and *completely disagree*, corresponding to scores from 5 to 1. The first question, *The project's task description is clear and precise*, assesses whether the generated content is readily understood by students. Since reasonable learning steps help create an effective learning experience, the second one is *The project has a reasonable setup with multiple learning steps*. The third question, *The project has improved my programming skills*, evaluates how much a project aids learners in programming learning. The fourth question, *The project is difficult for me*, assesses the project difficulty for students. The fifth question is *Rate the project quality* on a scale of 1–10. The reliability and validity of this questionnaire were measured. The fourth question reduced the questionnaire's validity below 0.7, so we removed it. Finally, the Cronbach's alpha was $0.726 > 0.7$, indicating an acceptable level of reliability. The Kaiser–Meyer–Olkin value was $0.718 > 0.7$ and the p -value in Bartlett's test was smaller than 0.001, showing independence between the various dimensions of the items.

1) *Results*: The questionnaire data are analyzed using IBM SPSS statistics, and the results are given in Table VI. According to the results of the Shapiro–Wilk test, most data have significance levels over 0.05, indicating that they do not meet normal distribution. Thus, a nonparametric test method, the Mann–Whitney U test, is used. For the first question, $p = 0.072$ for the Redis project, and $p = 0.911$ for the Match 3 game project, both

of which are above 0.05, showing no significant performance difference in the task description between the man-made projects and the LLM-generated ones. It also shows the efficacy of the IAM-CRBA, which enables the LLM to exert professional skills. For the second question, both p -values are greater than 0.05, confirming that the LLM can generate reasonable multiple steps that compare favorably with humans.

For the third question, more students believe that both projects can enhance their programming skills, reflecting that the LLM-generated projects have a good educational function. For the last question, the Redis project's p -value is below 0.05 and the LLM-generated one has a higher score. The match 3 game's p -value is above 0.05, indicating similar performances. With only two projects tested, it cannot be claimed that the LLM-generated projects are better than the other one. Our answer to the second research question: *What is the result of comparing LLM-generated projects using the proposed IAM-CRBA versus projects generated manually from the student's perspective?* is that the programming projects generated by the proposed IAM-CRBA have comparable performance to the man-made ones in terms of project description, learning step setting, assistance to students, and overall project quality.

VI. CONCLUSION, LIMITATIONS, AND FUTURE DIRECTIONS

In this study, we propose a CRBA that can be configured for different roles, executing tasks aligned with their respective strengths. Based on this, an iterative architecture of multi-CRBAs is implemented, guiding an LLM to criticize and refine its output automatically. The input of learner level and project requirements is supported to satisfy the lecturer's needs. By addressing the two research questions, it is demonstrated that different configurations of CRBA roles along with descriptions for the same role can produce different results. The choice of configurations depends on multiple facets, such as learner ability, teaching design, course objectives, etc. Through the user study, it is found that the generated projects have comparable performance to the man-made ones from the perspective of students.

This study has several limitations. First, the selection of experts was based on convenience sampling, which may limit the credibility and generalizability of our findings. The choice of ten experts was made to ensure a focused discussion, but this number might not capture the full diversity of perspectives that a larger or differently selected group could provide. Besides, due to the small-scale characteristic of the study, the insights gained are specific to this group of experts and may not be generalizable to a broader context. Similarly, the user study involved a relatively small convenience sample consisting of two programming projects and 60 undergraduates. The conclusion is applicable to certain educational scenarios in this article, yet its generalizability may be limited.

There are several directions for future research. First, it was observed during the experiments that the generated code becomes less reliable as project complexity rises and the number of learning steps increases. So, we will investigate the high-quality generation of more complex programming projects and also

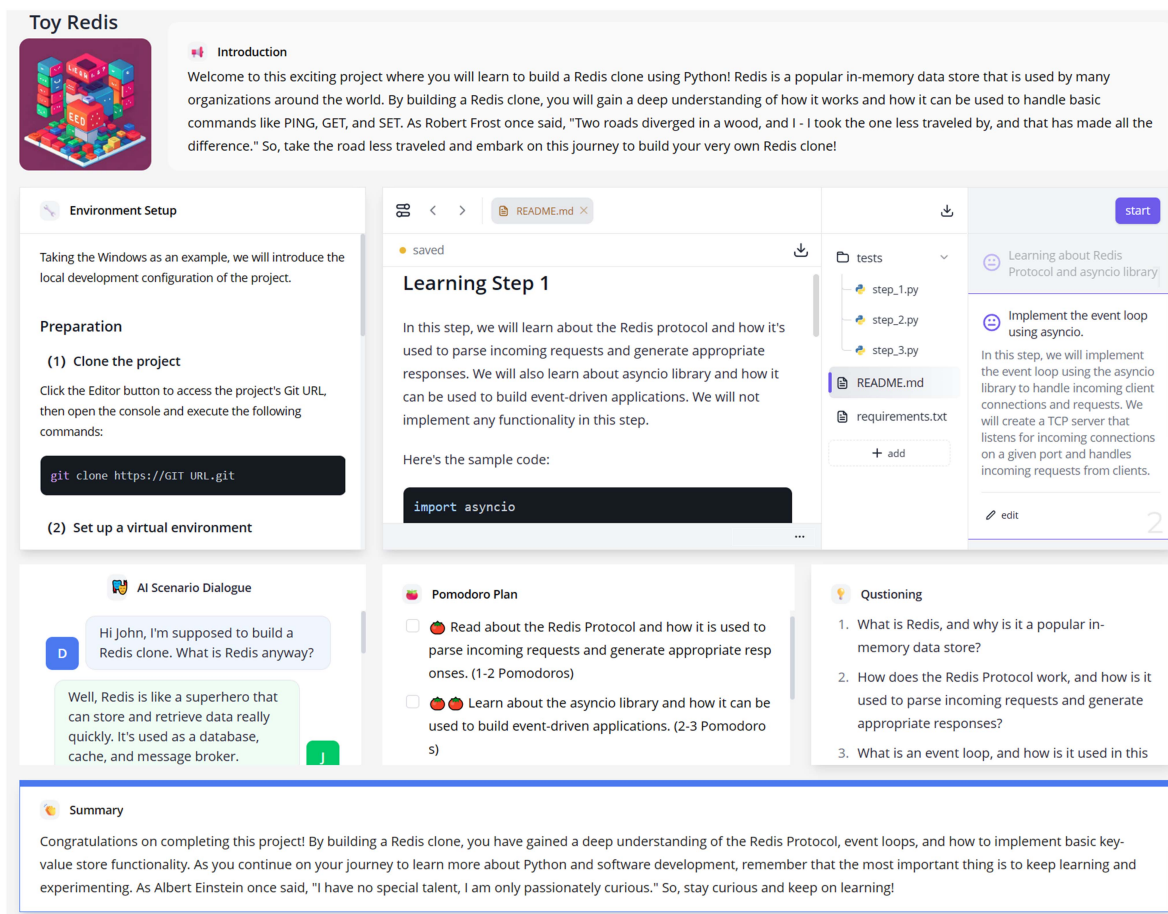


Fig. 14. User interface for presenting the generated project.

consider the inclusion of GPT-4, which offers greater computational power and improved capabilities for sophisticated tasks. Second, we will explore how to generate a personalized project for individuals to promote personalized learning and recruit more students to participate in our experiment. Third, we aim to expand the sample size, involving more experts, programming projects, and students with diverse layers, characteristics, and requirements.

APPENDIX A

A. User Interface

The user interface of the system for presenting the generated projects is shown in Fig. 14.

APPENDIX B

A. Details of Man-Made Projects

A computer science lecturer from the Beijing Institute of Technology was invited to create man-made programming projects. Tables I and IV and the following design guidelines are provided to the lecturer.

- 1) Projects should be aligned with the student level.
- 2) Projects should include multiple learning steps.

TABLE VII
MAN-MADE REDIS PROJECT

Introduction	Using the Go language to build a simple Redis database system that can perform operations such as setting, getting, and deleting key-value pairs. Through this process, you will learn the basic principles of Redis, as well as command execution and result processing.
Learning steps	1. Bind to local port 6379 (the default port used by Redis) and listen for TCP connections. 2. Respond to PING command with PONG. 3. Respond to multiple PING commands with PONG. 4. Handle concurrent connections. 5. Execute the ECHO command and return command parameters. 6. Execute SET and GET commands to save key-value pairs in memory. 7. Handle disconnection of clients.

- 3) Projects should encompass a certain number of knowledge points to facilitate learning.

Due to the limited space of this article, only the man-made Redis project is presented here, as given in Table VII.

REFERENCES

- [1] J. Sheard and K. Falkner, "Pedagogical approaches," in *The Cambridge Handbook of Computing Education Research*. Cambridge, U.K.: Cambridge Univ. Press, 2019, pp. 445–480.

- [2] H. Roumani, "Design guidelines for the lab component of objects-first CS1," *ACM SIGCSE Bull.*, vol. 34, no. 1, pp. 222–226, 2002.
- [3] A. Johnson, "Procedural memory and skill acquisition," *Handbook Psychol.*, vol. 4, pp. 499–523, 2003.
- [4] L. Yan et al., "Practical and ethical challenges of large language models in education: A systematic scoping review," *Brit. J. Educ. Technol.*, vol. 55, no. 1, pp. 90–112, 2024.
- [5] M. Chen et al., "Evaluating large language models trained on code," 2021, *arXiv:2107.03374*.
- [6] J. M. Wing, "Computational thinking," *Commun. ACM*, vol. 49, no. 3, pp. 33–35, 2006.
- [7] OpenAI, "Introducing chatGPT," 2022. [Online]. Available: <https://openai.com/blog/chatgpt/>
- [8] M. M. Rahman and Y. Watanobe, "ChatGPT for education and research: Opportunities, threats, and strategies," *Appl. Sci.*, vol. 13, no. 9, 2023, Art. no. 5783.
- [9] I. Balaban, B. Rienties, and P. H. Winne, "Information communication technology (ICT) and education," vol. 13, 2023, Art. no. 12318.
- [10] P. Sridhar, A. Doyle, A. Agarwal, C. Bogart, J. Savelka, and M. Sakr, "Harnessing LLMs in curricular design: Using GPT-4 to support authoring of learning objectives," *LLM@AIED*, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:259076210>
- [11] S. Sarsa, P. Denny, A. Hellas, and J. Leinonen, "Automatic generation of programming exercises and code explanations using large language models," in *Proc. 2022 ACM Conf. Int. Comput. Ed. Res.-Volume 1*, 2022, pp. 27–43.
- [12] P. Denny, S. Sarsa, A. Hellas, and J. Leinonen, "Robosourcing educational resources—leveraging large language models for learnersourcing," 2022, *arXiv:2211.04715*.
- [13] D. Leiker, S. Finnigan, A. R. Gyllen, and M. Cukurova, "Prototyping the use of large language models (LLMs) for adult learning content creation at scale," *LLM@AIED*, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:259076210>
- [14] E. Kasneci et al., "ChatGPT for good? On opportunities and challenges of large language models for education," *Learn. Individual Differences*, vol. 103, 2023, Art. no. 102274.
- [15] R. Dijkstra et al., "Reading comprehension based question answering system in Bangla language with transformer-based learning," *Heliyon*, vol. 8, no. 10, 2022, Art. no. e11052.
- [16] X. Lu, S. Fan, J. Houghton, L. Wang, and X. Wang, "ReadingQuizMaker: A human-NLP collaborative system that supports instructors to design high-quality reading quiz questions," in *Proc. 2023 CHI Conf. Hum. Factors Comput. Syst.*, 2023, pp. 1–18.
- [17] A. Blobstein, D. Izmaylov, T. Yifat, M. Levy, and A. Segal, "Angel: A new generation tool for learning material based questions and answers," in *Proc. NeurIPS'23 Workshop Generative AI Educ.*, New Orleans, LA, USA, 2023.
- [18] A. M. Olney, "Generating multiple choice questions from a textbook: LLMs match human performance on most metrics," in *Proc. AIED Workshops*, 2023, pp. 111–128.
- [19] F. Qu, X. Jia, and Y. Wu, "Asking questions like educational experts: Automatically generating question-answer pairs on real-world examination data," 2021, *arXiv:2109.05179*.
- [20] S. R. Piccolo, P. Denny, A. Luxton-Reilly, S. Payne, and P. G. Ridge, "Many bioinformatics programming tasks can be automated with chatGPT," 2023, *arXiv:2303.13528*.
- [21] E. L. Ouh, B. K. S. Gan, K. Jin Shim, and S. Wlodkowski, "ChatGPT, can you generate solutions for my coding exercises? An evaluation on its effectiveness in an undergraduate Java programming course," in *Proc. 2023 Conf. Innov. Technol. Comput. Sci. Educ. V. 1*, 2023, pp. 54–60.
- [22] K. Malinka, M. Peresini, A. Firc, O. Hujnák, and F. Janus, "On the educational impact of chatGPT: Is artificial intelligence ready to obtain a university degree?," in *Proc. 2023 Conf. Innov. Technol. Comput. Sci. Educ. V. 1*, 2023, pp. 47–53.
- [23] J. White et al., "A prompt pattern catalog to enhance prompt engineering with chatGPT," 2023, *arXiv:2302.11382*.
- [24] B. Reeves et al., "Evaluating the performance of code generation models for solving parsons problems with small prompt variations," in *Proc. 2023 Conf. Innov. Technol. Comput. Sci. Educ. V. 1*, 2023, pp. 299–305.
- [25] P. Denny, V. Kumar, and N. Giacaman, "Conversing with copilot: Exploring prompt engineering for solving CS1 problems using natural language," in *Proc. 54th ACM Tech. Symp. Comput. Sci. Educ. V. 1*, 2023, pp. 1136–1142.
- [26] Y. Shoham, "Agent-oriented programming," *Artif. Intell.*, vol. 60, no. 1, pp. 51–92, 1993.
- [27] A. Madaan, N. Tandon, D. Rajagopal, P. Clark, Y. Yang, and E. Hovy, "Think about it! improving defeasible reasoning by first modeling the question scenario," in *Proc. 2021 Conf. Empirical Methods Natural Lang. Process.*, Nov. 2021, pp. 6291–6310.
- [28] L. Ouyang et al., "Training language models to follow instructions with human feedback," *Adv. Neural Inf. Process. Syst.*, vol. 35, pp. 27730–27744, 2022.
- [29] D. Paul et al., "Refiner: Reasoning feedback on intermediate representations," in *Proc. 18th Conf. Eur. Chapter Assoc. Comput. Linguistics, Vol. 1: Long Papers*, Mar. 2024, pp. 1100–1126.
- [30] Y. Shi et al., "Leveraging GPT-4 for food effect summarization to enhance product-specific guidance development via iterative prompting," *J. Biomed. Inform.*, vol. 148, 2023, Art. no. 104533.
- [31] A. Madaan et al., "Self-refine: Iterative refinement with self-feedback," *Adv. Neural Inf. Process. Syst.*, vol. 36, pp. 46534–46594, 2024.
- [32] Z. Gou et al., "CRITIC: Large language models can self-correct with tool-interactive critiquing," in *Proc. 12th Int. Conf. Learn. Representations*, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:258823123>
- [33] A. Simões and R. Queirós, "On the nature of programming exercises," in *Proc. Int. Comput. Program. Educ. Conf.*, 2020. [Online]. Available: <https://api.semanticscholar.org/CorpusID:220055632>
- [34] A. Collins et al., "Cognitive apprenticeship: Teaching the craft of reading, writing, and mathematics," pp. 453–494, Dec. 2018, doi: [10.4324/9781315044408-14](https://doi.org/10.4324/9781315044408-14).
- [35] A. Vihavainen, M. Paksula, and M. Luukkainen, "Extreme apprenticeship method in teaching programming for beginners," in *Proc. 42nd ACM Tech. Symp. Comput. Sci. Educ.*, 2011, pp. 93–98.
- [36] F. K. Akin, "The art of chatGPT prompting: A guide to crafting clear and effective prompts," 2023. [Online]. Available: <https://fka.gumroad.com/l/art-of-chatgpt-prompting>
- [37] R. Lynch, A. Hurley, O. Cumiskey, B. Nolan, and B. McGlynn, "Exploring the relationship between homework task difficulty, student engagement and performance," *Ir. Educ. Stud.*, vol. 38, no. 1, pp. 89–103, 2019.
- [38] M. Konecki, "Problems in programming education and means of their improvement," *DAAAM Int. Sci. Book*, vol. 2014, pp. 459–470, 2014.
- [39] S. Küchemann et al., "Physics task development of prospective physics teachers using chatGPT," 2023, *arXiv:2304.10014*.
- [40] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*, 1st ed. Upper Saddle River, NJ, USA: Prentice Hall, 2008.



Tian Song (Member, IEEE) received the M.Eng. and Ph.D. degrees in computer science from Tsinghua University, Beijing, China, in 2005 and 2008, respectively.

He is currently a Professor with both the School of Education and the School of Cyberspace, Beijing Institute of Technology, Beijing. His current research interests include AI and education, online courses, network security, and Internet architecture.



Hang Zhang received the M.S. degree in computer science and technology from the Beijing Institute of Technology, Beijing, China, in 2016.

He is currently a Researcher with the School of Education, Beijing Institute of Technology. His research interests include the application of AI-powered educational technology and AI supported knowledge graph generation.



Yijia Xiao received the bachelor's degree in communication engineering from the Communication University of China, Beijing, China, in 2022. She is currently working toward the master's degree with Beijing Institute of Technology, Beijing.

Her main research interests include the application of artificial intelligence and Large Language Models.